



OASIS Security Report

Insecure Deserialization Security Analysis

Date: 2026-06-11 10:12:05 — **Model:** mistral:latest

Vulnerability: Insecure Deserialization

Project: clean_contracts

Analysis root: ../clean

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **3** file(s) for Insecure Deserialization.

[Return to table of contents](#)

Statistics

Files analyzed: 3 **Total findings:** 3

Critical		0
High		0

Medium

2

Low

1

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/clean/simple_dao_fixed.sol</code>	0.647
<code>/work/project/dataset/clean/modifier_reentrancy_fixed.sol</code>	0.646
<code>/work/project/dataset/clean/mycontract_fixed.sol</code>	0.597

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.647

Source lines: 1-34

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
credit[msg.sender]-=amount; require(msg.sender.call.value(amount));
```

Explanation

The use of `msg.sender.call` can lead to deserialization vulnerabilities if the sender's contract is malicious and contains code that can manipulate data.

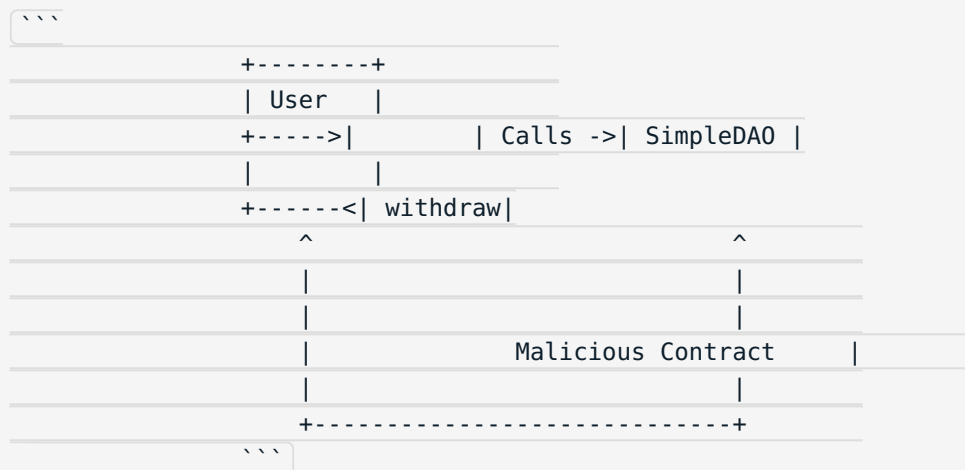
Impact

Denial of Service, Data tampering

Entry point

withdraw function

Execution path



Parameters

amount

Exploitation steps

- User sends a transaction to withdraw funds
- Malicious contract intercepts the call
- The malicious contract sends malicious data during deserialization

Conditions

An attacker must have control over a user's contract.

Remediation

Implement safe deserialization methods or validate object types when handling user input.

Secure example

```
// Implement a safe deserialization method or validate the data before calling  
the function  
require(msg.sender.call.value(amount).safeCall());
```

File 2: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.646

Source lines: 1-42

► Finding 2.1: Vulnerability found (Medium)

Vulnerable code

```
function supportsToken() external returns(bytes32)
{ return(keccak256(abi.encodePacked("Nu Token"))); }
```

Explanation

Function deserializes user input (keccak256(abi.encodePacked("Nu Token"))) without validation, making it vulnerable to Insecure Deserialization.

Impact

Arbitrary code execution or data tampering

Entry point

Bank.supportsToken() function

Execution path

```
'''
user -> Bank.supportsToken() -> deserialize user input
(keccak256(abi.encodePacked("Nu Token"))) -> code execution
'''
```

Parameters

input to supportsToken function

Exploitation steps

- Provide custom input to deserialize
- Exploit Insecure Deserialization to execute arbitrary code

Conditions

Ability to provide input to the supportsToken function

Remediation

Validate user input before deserializing, or use a safe deserialization method.

Secure example

```
// Safe example
function supportsToken(bytes32 _token) external returns(bool) {
    require(_token == keccak256("Nu Token"), "Incorrect token supplied.");
    return true;
}
```

File 3: /work/project/dataset/clean/mycontract_fixed.sol

Similarity score: 0.597

Source lines: 1-31

► Finding 3.1: Vulnerability found (Low)

Vulnerable code

none

Explanation

The contract does not handle user input, so it does not have an insecure deserialization vulnerability.

Impact

None (contract does not accept or deserialize user data)

Remediation

None (contract does not accept or deserialize user data)

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)